

Progress report - Ghost_Protocol

TEAM NAME: Ghost_Protocol | PROBLEM STATEMENT 5 | DATE: 08 SEPTEMBER 2026

1. Name of the Problem Statement

Quantum-Inspired Cyber Threat Detection Framework for Teleportation-Based Quantum Digital Signature Security

2. Abstract of Overall Proposed Idea

Quantum Digital Signature (QDS) protocols provide information-theoretic security against quantum algorithms such as Shor's algorithm, which compromise classical public-key cryptography. This project presents a non-machine-learning, quantum-inspired cyber threat detection framework tailored for teleportation-based QDS systems. The architecture integrates a classical pre-processing module utilizing SHA-256 hashing and 256-bit secret key XOR encoding with a 3-qubit quantum teleportation pipeline. Alice prepares Pauli eigenstates across a deterministic Z/X/Y basis schedule, transmitting them via Bell-state EPR pairs and conditional Pauli corrections. Bob performs projective measurements to verify signature authenticity against calibrated baseline channel noise (p_0). The core threat detection engine replaces artificial intelligence with an exact Binomial upper-tail hypothesis testing model ($P(K \geq k \mid n, p_0)$). The framework detects five physical threat vectors: channel tampering (Pauli-X bit-flips), signature forgery ($K=0$ assumption), impersonation (Bernoulli guessing), quantum interception (intercept-resend measurement collapse), and replay attacks (digest Hamming distance analysis). Verified on Qiskit AerSimulator and validated on physical IBM Quantum QPUs via Qiskit Runtime, the system guarantees deterministic acceptance of legitimate signatures, low computational complexity, and robust security guarantees.

3. Work Done / Implemented Till 8 PM Today (08/09/2026)

Fully implemented end-to-end QDS protocol, exact Binomial threat detector, 5 physical attack models (channel tampering, forgery, impersonation, interception, replay), 70-test regression suite, Streamlit research laboratory UI, 256-qubit experimental trace inspector, and optional IBM Quantum QPU integration with Streamlit Cloud deployment.

4. Work Left

Phase 6 enhancements: integrating session nonces for same-message replay prevention, benchmarking hardware execution latency across multiple IBM QPUs under varying queue depths, expanding key agreement protocols for multi-recipient verification, and generating comparative benchmarking reports against classical ECDSA signatures.

5. Challenges Faced

1. Quantum Measurement Collapse & Basis Invariance Handling: Designing physical attack models (channel tampering, intercept-resend) without fabricating error rates required precise simulation of quantum eigenstate projections. Handling basis-wise invariance ($X|+\rangle = |+\rangle$) while capturing state collapse during out-of-basis Eve measurements required custom Qiskit circuit pipelines.

2. Hardware API & UI Execution Compatibility: Integrating optional physical IBM Quantum QPU execution (qiskit-ibm-runtime) alongside local AerSimulator required handling account channel differences (ibm_cloud vs ibm_quantum), managing non-blocking job execution, and resolving cloud dependency requirements (pylatexenc) while ensuring zero exposure of sensitive API keys.

Deliverables Verification Matrix (Codebase Mapping)

S.No	Expected Deliverable	Key Components / Metrics	Codebase Implementation & File Mapping
1	Mathematical Model of Teleportation-based QDS	Bell-state entanglement, 3-qubit teleportation, Pauli corrections ($Z^{c_0}X^{c_1}$), 6 Pauli eigenstates	• qds/states.py: 6 Pauli eigenstates, Z/X/Y basis matrices. • qds/teleportation.py: 3-qubit Bell pair, Pauli corrections. • qds/circuit_visualization.py: Statevector & Bloch formulas.
2	Quantum-Inspired Threat Detection Framework	Non-ML core detection engine for forgery, impersonation, replay, and channel tampering	• statistics/detector.py: Exact Binomial upper-tail test ($P(K \geq k n, p_0)$). • evaluation/runner.py: Comparative security sweeps.
3	Signature Generation & Verification Module	QKD simulation, Signature generation, Verification algorithm with Pauli corrections	• qds/encoding.py: SHA-256 digest (D), XOR key mixing ($b_i = d_i \oplus K_i$), basis schedule. • qds/verification.py: Bob-side projective measurement readout. • core/backend.py: AerSimulator adapter.
4	Attack Simulation Module	Simulation of cyber threats: Forgery, Impersonation, Replay, Channel Tampering	• attacks/channel.py: Pauli-X channel noise. • attacks/forgery.py: Digest-only forgery ($K=0$). • attacks/impersonation.py: Bernoulli(0.5) guessing. • attacks/interception.py: Intercept-resend collapse. • attacks/replay.py: Replay & Hamming distance.
6	Software Framework / Prototype	Simulation environment, Verification interface, Threat detection dashboard, Event logging	• app.py: Streamlit web dashboard with 7 sections, 256-qubit outcome maps, Qiskit drawer. • core/hardware.py: Real IBM Quantum QPU execution. • tests/: 70 unit tests, 100% passing.